Hash-algorithms Output for Digital Evidence in Computer Forensics

Hung-Jui Ke,
[1] Information Management Office,
Changhua County Police Bureau,
Changhua County, Taiwan 500

Shiuh-Jeng Wang*[,2],
[2] Department of Information Management,
Central Police University,
TaoYuan, Taiwan 33304
sjwang@mail.cpu.eud.tw
*Correspondence

Jonathan Liu[3],
[3] Department of Computer, Information Science and EngineeringUniversity of Florida, USA

Dushyant Goyal[4]
[4] Department of Electronics and Communication EngineeringLNM Institute of Information Technology, Jaipur, India

Abstract

With the fast developments in computer and internet connections going through the world, the high-tech crime problem is also companying with it on the way. Hash-algorithm is, basically, a crypto-function that enables to output a short-string for any possible input digital strings. One of its applications is able to be used in the field of computer forensics to authenticate the completeness of digital evidence. According to the collision-free nature of hash, any one minor change of strings in function input, the output-strings will be totally diverse. In this paper, we review the hash property and discuss the related applications in computer forensics to enhance the reliability of digital evidence in the investigation of computer crime-events

Keywords: Hash, Authentication, Digital Evidence, Computer Forensics, Cyber-crime

1. Introduction

Recently, computer forensics is more and more well-known in the world due to its varied applications. Computer Forensics is considered as an effective way to crack computer crimes based on the standard procedure: preservation, examination, analysis, and presentation [9].

i. Preservation: The first thing to do on field is to preserve the evidence. Digital evidence is very fragile. So, we need to control the scene and document the time and operation. Sometimes, we need to duplicate entire hard disk by using bit stream copy. Bit string copy can make sure that the source data and the duplicated data are the same. The hash function is usually used to authenticate the integrity of the data.

ii. Examination: We use lots of utility software to examine the evidence. The evidence could contain images, videos, audio and text-files. We also need to examine the slack space to probe the deleted files.

iii. Analysis: After analyzing the evidence, we categorize, compare and individualize the interactions between the suspects and the evidence. In other words, we need to connect the evidence, suspects, victims and crime scene by following the evidence [7, 9], as shown in Fig. 1.

iv. Presentation: When we find out the connections between the suspects and evidence, we have to present the clear and undoubted result in court. Besides, the alternative reasons must be ruled out and to definitely prove the uniqueness of the digital evidence we present.

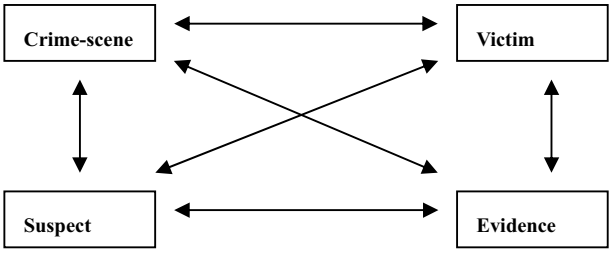

Fig. 1 The connected-links among the evidence, suspects, victims and crime scene

In the step of preservation and examination, the hash function is commonly used to authenticate the consistency between the original evidence and the duplicated one [5]. In 2004, one of the hash-algorithms, MD5 (Message Digest 5) had been cracked in cryptanalysis. Soon after this event, it was then announced that the SHA-1 (Secure Hash Algorithm) had been cracked as well [11, 12]. This series of information shocked the researches in the field of cryptography, and raised the attention of every related organization of security application mechanisms. Because the hash function is the foundation of computer forensics, it is therefore inspired to conduct this paper for the

978-0-7695-4532-5/11 $26.00 © 2011 IEEE
DOI 10.1109/BWCCA.2011.65

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.

developments of a variety of hash functions so as to still sustain its soundness in the completeness of digital evidence in the course of investigations upon computer forensics.

The presentation of this paper is organized as follow. Brief history, development and applications of Hash-algorithms are discussed in Sec. 2. In the same section, the most popular hash functions MD5 and SHA-1 are also introduced. The Hash-algorithms adopted in computer forensics are discussed in Sec. 3. Countermeasures for hash-algorithm developments are presented in Sec. 4. Finally, we conclude our paper in Sec. 5.

2. Hash challenge and Applications on computer forensics

A hash function is a function that takes an arbitrary amount of input and produces an output of fixed size. All hash functions have the property that it is impossible to determine the input knowing only the output. The properties of some hash functions can be used to greatly increase the security of a system and provide integrity to a source of information/data. In this section we will examine the history and security available in various hash implementations. The cryptographic algorithms mentioned are all based on cryptographic hash functions and these hash functions can be combined with other standard cryptographic methods to verify the source of data.

Description of a good hash function can be summarized to have the following properties:

- Collision resistance**:** It is computationally infeasible to find x and y where *x != y and H(x) = H(y)*
  That is, it is impractically difficult to find a *random collision.* Two distinct messages that result in the same digest are called collisions
- Preimage resistance**:** Given a desired output y, it is computationally infeasible to find x where *H(x) = y*
  That is, it is impractically difficult to find a *chosen-target collision.*
- Second pre-image resistance**:** Given an input x', it is computationally infeasible to find x where *H(x) = H(x')*
  That is, it is impractically difficult to find a *chosen-target collision.*
- Avalanche Effect: Given two inputs x1 and x2 differing slightly among themselves, the hash function should be able to find H(x1) and H(x2) which differ drastically (Fig. 2).

where H(.) corresponds to the cryptographic hash function, x and y represent the data values to be hashed.

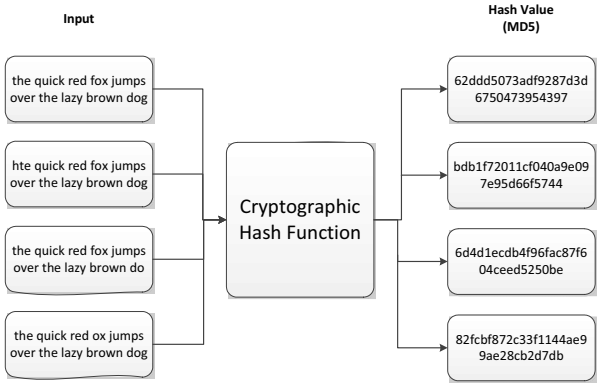

Fig. 2 An example of MD5 algorithm

MD5 and SHA-1, the most popular cryptographic hash functions have been used to authenticate the integrity of the evidence as for the secure and quickness. Because the algorithms of MD5 and SHA-1 are cracked, we need to examine the impact on computer forensics. Let us first give a brief review of the hash study with history, development and applications in security.

2.1.1 MD5 [3]

In 1991, Rivest designed MD5 (Message Digest 5) as a hash-algorithm. It became one of the most famous algorithms for the implementation of hash. MD5 can processes a variable length message into an output of fixed-length 128-bit. The output of hash is so-called as digest (message digest). The basic process to output the digest in the case of MD5 is briefly listed as follows.

i. Divide an input message of any length by 2^{64} and obtain the residue.
ii. Check the input message to see if the length is congruent to 512 modulo 448. If not, add a sequence of bits as required.
iii. Vacate 4 four-word buffers (A,B,C,D) as initial buffer to compute the message digest.

This completes the description of MD5. The dimension of security is about 2^{64}, meaning that it is supposed to be calculated 2^{64} times to get the same digest.

To check the result we use the standard test in the RFC1321, as provided by Rivest, and shown in Table 1. See the first input message. If the input message is null, then the output format is in hexadecimal number as **d41d8cd98f00b204e9800998ecf8427e**.

Table 1 Benchmark-sample of RFC 1321 for MD5 [3]

Message	Digest
Null	d41d8cd98f00b204e9800998ecf8427e
'a'	0cc175b9c0f1b6a831c399e269772661
'abc'	900150983cd24fb0d6963f7d28e17f72
'message digest'	f96b697d7cb7938d525a2f31aaf161d0
'abcdefghijklmnopqrstuvwxyz'	c3fcd3d76192e4007dfb496cca67e13b

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.

2.1.2 SHA [2]

SHA (Secure Hash Algorithms) was published in 1993 by the National Security Agency (NSA) and the National Institute of Standards and Technology (NIST). SHA-1, the most famous successor of SHA was proposed in 1995. The process of digest of SHA-1 is similar to MD5. The differences between SHA-1 and MD5 are the output bits length and initial buffers. SHA-1 produces the length of digest is longer than MD5, so it is considered more secure than MD5. It is supposed to be calculated more 2^{80} times to have the same digest for the different input message. In Table 2 we use the description which was published by NIST as a further explanation.

Table 2 Benchmark for SHA-1 [2]

Message	Digest
'abc'	a9993e364706816aba3e25717850c26c9cd0d89d
'abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq'	84983e441c3bd26ebaae4aa1f95129e5e54670f1
a million of 'a'	34aa973cd4c4daa4f61eeb2bdbad27316534016f

The standard benchmark is used to check the correctness of hash program. We use *H( )* as hash function, *M* as the original message. The output of *H(M)* is a digest. Assume that there is a different message M'. Whenever *H(M)* totally matches *H(*M'*),* it means that, *M* must be equal to M'. Because either *H(M)* or *H(*M'*)* is a fixed-length bit-string, there is no clue to know the input message. In other words, it is computationally impossible to recover the input message *M* or M' from the digest *H(M)* or *H(*M'*)* . The security of MD5 and SHA-1 are thought as reliability over the past time that is due to the digest collision is lower than 2^{-64}. It is required to take a long time to crack the inconsistency for the digests in two different input messages.

Any kind of cryptographic algorithms always come to meet the risk of being cracked. If a cryptosystem would take 100 years to be cracked, we can estimate that it can be broken in approximately 20 days when we have the speed of a computer system was to be 2000 times faster. An unbreakable algorithm simply does not exist, because any algorithm can be attacked by simple brute force. We only focus on the computational security of these algorithms.

The cracking of MD5 or SHA-1 does not mean they can recover the original message soon. It is to prove that a faster way was found for the output message digests which are the same for those different input messages in a reasonable time. So MD5 and SHA-1 were claimed not computationally secure any more. The latest report shows the security of SHA-1 should be downgraded from the probability of 2^{-80} to 2^{-69}. It means that it should be reduced about 2000 times. As matter of fact, the probability number of 2^{-69} is still enormously small. It is still a milestone to the hash cryptanalysis. In such a way, the same message digest produced for different inputs by MD5 or SHA-1 is possible in reasonable time nowadays [10].

2.1.3 Tiger [4]

Tiger is a cryptographic hash function designed by Ross Anderson and Eli Biham in 1995. In general, the size of a Tiger hash value is 192 bits, but several other truncated versions such as Tiger/128 and Tiger/160 can also be used for compatibility with protocols with particular hash sizes. Tiger is designed using the nearly universal Merkle-Damgård paradigm.The one-way compression function operates on 64-bit words, maintaining 3 words of state and processing 8 words of data. There are 24 rounds and a combination of operation with XOR, addition/subtraction, rotation, S-box lookups, and a fairly intricate key scheduling algorithm are used for deriving 24 round keys from the 8 input words [1].

Tiger has no usage restrictions or patents and can be used freely with the reference implementation.

The output printing method for the hash is big endian byte order. The following examples in Table 4 demonstrate some ASCII inputs and the corresponding Tiger hashes.

Table 4 Benchmark for Tiger

Message	Digest
'abc'	25285ffcc8838853fdf4bd668e0199025317e9f2657befb5
'Hello World'	c96eec5a4121cf48aaee6949b9339f1b09f20c5e5eda4388
Null	4635fff6a778cc243da15c69594e98e79451256e680b4e80

2.2 Applications of Hash functions

There are an abundant number of applications of hash functions in cryptography and computer forensics. In this section we will briefly describe several scenarios, which can be used as models for assessing security of real-life applications.

2.2.1 Digital signatures

Digital signatures are being used since a long time and were the first application of cryptographically secure hash functions. Hash functions serve a role for demonstrating the authenticity of a digital message or document. They are commonly used for detecting forgery or tempering in applications like software distribution and financial transactions. Message Authentication Code is a short piece of information (tag) used to authenticate a message. The MAC protects both a message's data integrity and authenticity by allowing verifiers to detect any changes to

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.

the message content. When hashing algorithms are combined with encryption, they produce special message digests that identify the source of the data; these special digests are called Message Authentication Codes. The standard algorithm currently used today is called HMAC. The HMAC algorithm provides verification of the source of data, and also prevents against attacks such as the replay attack.

Lastly there is a discussion of how the two related fields of encryption and hash functions are complementary, not replacement technologies for one another.

2.2.2. Password tables for client authentication

In order to authenticate a client, he/she is required to enter a password previously registered with the server. But storing the passwords of all users on the server poses an obvious security risk. With the use of hash functions the server can instead store the non-invertible and collision free hash values and use this information to match with the hash of the alleged password presented by the client.

2.2.3 Cyclic Redundancy Check

CRC is a non-secure hash function technique for detecting errors in digital data transmission. It is also applied to data storage devices, such as disk drives, etc. where each block on the disk have check bits and the hardware automatically initiates a reread of the block when an error is detected or reports the error to software.

2.2.4 Key derivation

Key derivation function (KDF) derives one or more secret keys from a secret value such as a master key or other information such as a password using a pseudo-random function. KDFs are used in conjunction with non-secret parameters to derive one or more keys from a common secret value (key diversification).Sharing secret may prevent an attacker from learning the secret information.

2.2.5 GUID

Globally unique identifier (GUID) is a unique reference number used as an identifier in computer software. Some of the common uses are Microsoft's implementation of Universally Unique Identifier (UUID) standard, Intel GUID Partition Table for partitioning hard drives, etc.

3. Hash applied in computer forensics

As we mentioned above, hash takes an important part in computer forensics. Computer forensics experts use hash function to test the integrity of evidence because it is easy and fast. So, hash function exists in a number of forensic toolkits (or audit toolkits), such as Encase, TCT (The Coroner's Toolkit), Sleuthkit and Tripwire. The basic hash-algorithms in the current toolkits of computer forensics are always the hashes of MD5 and SHA-1 [8]. According to the one-way and collision-free properties of hash, in case we can obtain same digest from different input messages, it means that there are highly likely to be something wrong in the operations of forensic toolkits. Let us take more looks at the utilizations of hashes used in computer forensics as follows:

i. Computer forensic experts, in general, use hash function to digest files as alternative evidence.
ii. The message digest is used to compare the difference of the original files and duplicated ones. The digest is not the ONLY evidence.
iii. If the same digest happens for the input of different files, we always take the original file ones as the critical evidence in computer forensics.

The hash function used in forensic toolkits is usually to compare the output digest for the input files or messages. It is almost impossible to trace back the original data content through the digest, even though the collision probability has been raised by the hash cracking.

There is an issue of the altered evidence we need to pay attention here. In case someone claims that the same digest output means the evidence being altered, the suspect is framed on purpose in this way. At this point, we have to know that the digest representation is not the only one evidence. Besides, the evidence is managed through a legitimate procedure of *chain-of-custody* in computer crime investigations [6]. The following key features with 5-*W*-2-*H* are essences of inspections for data analysis in *chain-of-custody*.

i. *Who* collected it?
ii. *How* and *Where*?
iii. *Who* took possession of it?
iv. *How* was it stored and protected in storage?
v. *Who* took it out of storage and *Why*?

In criminal investigation any evidence collection, its transfer and processing is required to be recorded in order to maintain the completeness in the chain of custody. The record of the chain of custody is used to keep the evidence untainted. On the other hand, it is not easy to have the readable altered file which has the same digest with the original one, because the file content is not in simple few words.

For example, we took two cases to explain how hash works in forensics:

Case 1:

We use AIR (Automated Image & Restore) to duplicate seized disk as image file and verify with hash. Any investigator would sign his name with hashing code before delivering to the next investigators. Then, we mount the image file to analyze with Autopsy. We can see that the hashing codes of the example are the same. So, we can tell the evidence is not tampered. If the hashing codes do not match, we can trace to the one who should be blamed.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.

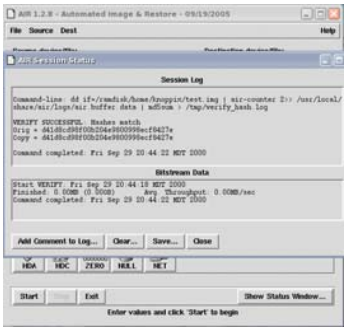

Fig. 3 The screenshot of AIR

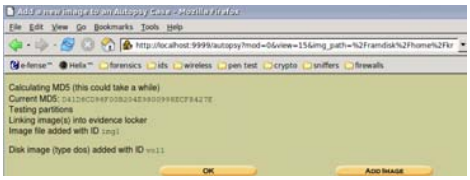

Fig. 4 Using MD5 hash Algorithm in Autopsy

Case 2:

When we do not need to duplicate entire seized disk drive, we also use the hash to calculate the files of evidence. We can use MD5 to calculate the suspicious files and produce one list with full path name and hashing code. Then, we take the hashing code of this list as a signature. If one file has altered, the hashing codes of this file and the list are also changed. The investigators have to sign his name and time while the evidence is transferred. So, when the evidence is challenged, we can trace the time stamp of files to know who was holding the evidence.

Fig. 5 Using MD5 hash Algorithm for files

Any action we performed on evidence should be updated in document to maintain the chain of custody. The document is also signed by police officers and investigators. When we can prove the chain of custody is not compromised, the evidence should be solid in the court of law.

4. Countermeasures

Although some of hash-algorithms, such as MD5 and SHA-1, have been proposed in the possibly cracked in a reasonable time. It is believed that the related hash-family does still remain rather high collision-free to resist the cryptanalysis. In the case, next, we present the salted hashes and sophisticated hashes to be the fundamentals of hash sustaining in computer forensics.

4.1 Salted hashes

As we know, the hash-algorithm has initial buffer as part of operation. We also can give a string as initial value to have different digest from the same message by hash function. The initial value is so called salt. The salts of hash, such as HMAC-MD5 and HMAC-SHA-1 can give more protection to the original content. Since the salt is incorporated into the hash procedure, we can use time stamp as salt to be a secret key. In this way, we can not only reduce the collision of hash but also strengthen the evidence reliability. The case of salted hash with input-string 'abc' is shown in Table 6.

Table 6 Case result of salted hash

Hash	Input	Salt	Digest
HMD5	'abc'	Null	**dd2701993d29fdd0b032c233cec63403**
HMAC-MD5	'abc'	2011-06-27 17:00	**958cf445652739943ab669dc2d561c36**
HMAC-MD5	'abc'	2011-06-28 17:00	**cfb5eaff17087cbb17912c9f2b5201ae**
HMAC-SHA-1	'abc'	null	**9b4a918f398d74d3e367970aba3cbe54e4d2b5d9**
HMAC-SHA-1	'abc'	2011-06-27 17:00	**9e631272273701b218c0125dc45a25bd68c8a244**
HMAC-SHA-1	'abc'	2011-06-28 17:00	**5adea38a5d4ad631cd64dd298dfd42c12ea43839**

4.2 A variety of hash-algorithms

Since, the chance of collision has been raised in the event of MD5 and SHA-1 being cracked, we have to seek another one aiming at the complete-proof of digital evidence, instead. Fortunately, there are already some substitutes proposed such as SHA-512 and Tiger. They are more difficult to break than SHA-1, and have been considered to be the candidate of new standard. In August 2002, for example, NIST released a series of new algorithms including SHA-224, SHA-256, SHA-384, SHA-512, and Tiger, and made them into the FIPS. The hash-algorithm of Tiger which was proposed by Anderson et al, the digest of Tiger is longer than SHA-1, which is 192-bit-long [4]. Besides, the SHA-512 can produce a digest of 512-bit. That means 2^{256} operation is required to produce the same digest by SHA-512. It is more difficult to obtain the same digest output for any two different string inputs. The list of the

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.

result of the testing for the string of 'abc' in the newer hash algorithms, are shown in Table 7.

Table 7 Result list of a string 'abc'

Hash	Digest
SHA-224	**23097d223405d8228642a477bda255b32aadbce4bda0b3f7e36c9da7**
SHA-256	**ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad**
SHA-384	**cb00753f45a35e8bb5a03d699ac65007272c32ab0eded1631a8b605a43ff5bed8086072ba1e7cc2358baeca134c825a7**
SHA-512	**ddaf35a193617abacc417349ae20413112e6fa4e89a97ea20a9eeee64b55d39a2192992a274fc1a836ba3c23a3feebbd454d4423643ce80e2a9ac94fa54ca49f**

The research team steered by X. Wang et al. [11, 12] have found that the higher collision ratio problems of hash. It definitely leads the researchers to invent more secure algorithms to strengthen hash structures. It seems similar to the AES that comes after the DES after receiving a lot of comments to stimulate and promote its security level.

5. Conclusion

We had assumed the security of MD5 and SHA-1 sound enough for many years. The MD5 and SHA-1 has therefore been applied to many fields because of their reliability and quickness. The research teams from China and the U.S.A. found that SHA-1 is thousands of times weaker than thought. The result of their research may not have the instant threat on information security, but the fact of "breaking" at the implementations of hash, such as MD5 and SHA-1 still shocks many people. It also forces us to think about the hash applications to computer forensics. The evidence of computer crime is crucial to show the reconstruct of the crime scene so as to unveil who is the guilty or innocent. The countermeasure to enhance the hash applications in evidence reliability is therefore prominent as well. In this paper, we analyze the basis property of hash and explain how secure it is in the view of cryptography. The cracking of MD5 and SHA-1 may not have immediate danger in the usage of computer forensics. Nevertheless, the substitute algorithms are advised to take over so as to mitigate the intimidation in cryptanalysis. It can be a variety of hash-family, such as salted hash, SHA-224, SHA-256, SHA-384, and SHA-512. The cryptanalysis threat for the top hash algorithms, MD5 and SHA-1, is not the doomsday at hash studies. We still summarize the one-way hash-family which is able to keep safe in evidence reliability in computer forensics. The hash-algorithms are, normally, devoted to make the probability of collision decrease as much as possible. In the next developments, how to fulfill the anticipations will be the prospective views to those who are interested in cryptography, computer security applications, and the computer forensics as well.

Acknowledgements: This research was partially supported by the National Science Council of the Republic of China under the Grant NSC 98-2221-E-015-001-MY3-, and NSC 99-2918-I-015-001.-

References

[1] CSRC Cryptographic Toolkit, http://csrc.nist.gov/groups/ST/toolkit/index.html

[2] FIPS 180-2, Secure Hash Standard (SHS), August 2002, http://csrc.nist.gov/publications/fips/fips180-2/fips180-2withchangenotice.pdf

[3] RFC 1321 - The MD5 Message-Digest Algorithm, http://www.ietf.org/rfc/rfc1321.txt?number=1321

[4] Tiger:A Fast New Cryptographic Hash Function (Designed in 1995), http://www.cs.technion.ac.il/~biham/Reports/Tiger/

[5] M. Karyda and L. Mitrou, "Internet Forensics: Legal and Technical Issues," Second International Workshop on Digital Forensics and Incident Analysis, 2007.

[6] G. Kruse II and J. G. Heiser, Computer Forensic: Incident Response Essentials, Addison Wesley, 2002, pp:2-8,163-174.

[7] H.C. Lee, T. Palmbach and M.T. Miller, Henry Lee's Crime Scene Handbook, Academic Press, 2001.

[8] S.M. Oh, Y.M. Kim, J.H. Jang, B.S. Koh, and Y.R. Choi, "A Study of Volatile Information Collection of Computer Forensics System for Computer Emergency based on Ubiquitous Computing," Third International Conference on Natural Computation, 2007.

[9] S.J. Wang, "Measures of Retaining Digital Evidence to Prosecute Computer-based Cyber-crime," International Journal Computer Standards & Interfaces, Vol. 29, No. 2. pp. 216-223, Jan., 2007.

[10] S.J. Wang, H.J. Ke, J.H. Huang, and C.L. Chan, "Hash Cracking and Aftereffect on Authentication Procedures in Cyberspace," IEEE Aerospace & Electronics Systems Magazine, Vol. 22, Issue 1, pp. 3-7, Jan. 2007.

[11] X. Wang, Y. L. Yin, and H. Yu, Finding Collisions in the Full SHA-1, Advances in Cryptogy—CRYPTO 2005, LNCS Volume 3621, 2005.

[12] X. Wang and H. Yu, How to Break MD5 and Other Hash Functions, Advances in Cryptogy—Eurocrypt 2005, LNCS Volume 3494, 2005.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:53 UTC from IEEE Xplore. Restrictions apply.